

Web Application Penetration Test Report

Damn Vulnerable Web Application (DVWA)

Assessment Type	Black-Box Web Application Penetration Test
Methodology	OWASP WSTG v4.2 · PTES
Vulnerabilities Covered	IDOR, SQL Injection, XSS, Broken Authentication
Classification	Portfolio / Educational Report
Prepared By	Ahmed Khader Metwally

1. Executive Summary

This report documents a structured penetration test conducted against DVWA, a deliberately vulnerable PHP/MySQL web application used for security training. The assessment followed the full penetration testing lifecycle — reconnaissance, exploitation, and reporting — with each vulnerability mapped to the OWASP Top 10 (2021) and documented with proof-of-concept evidence, impact analysis, and remediation guidance.

Four vulnerability classes were identified and successfully exploited: **Insecure Direct Object Reference (IDOR)**, **SQL Injection**, **Cross-Site Scripting (XSS)**, and **Broken Authentication**. Each finding independently demonstrates a critical security control failure with real-world exploitation potential.

2. Scope & Methodology

In-Scope Modules

SQL Injection, XSS (Reflected/Stored), Insecure Direct Object Reference, Authentication/Session Management

Testing Phases

Phase	Activity
1. Reconnaissance	Application crawling, endpoint enumeration
2. Scanning	Manual parameter analysis, Burp Suite baseline scan
3. Exploitation	Manual exploitation with documented payloads
4. Impact Validation	Confirming real-world exploitability and impact
5. Reporting	CVSS scoring, OWASP mapping, remediation guidance

3. Risk Summary

ID	Vulnerability	OWASP Category	Severity
DVWA-01	Insecure Direct Object Reference (IDOR)	A01:2021 - Broken Access Control	Critical
DVWA-02	SQL Injection	A03:2021 - Injection	Critical
DVWA-03	Cross-Site Scripting (XSS)	A03:2021 - Injection	High
DVWA-04	Broken Authentication	A07:2021 - Identification & Authentication Failures	High

4. Findings

Finding 1 — Insecure Direct Object Reference (IDOR)

Severity: Critical

OWASP Category: A01:2021 – Broken Access Control

Description

The application exposes internal object references (such as user or record IDs) directly in request parameters without verifying that the requesting user is authorized to access the referenced resource.

Proof of Concept

```
Original request:
GET /profile?id=5

Modified request:
GET /profile?id=6
```

The modified request returned data belonging to a different user account, confirming the absence of an authorization check on the server side.

Impact

An attacker can enumerate object IDs to access other users' data or restricted resources without any privilege escalation, resulting in unauthorized data exposure across the application.

Remediation

- Enforce server-side authorization checks on every object access, independent of client-supplied IDs.
- Use indirect reference maps (e.g., session-scoped tokens) instead of exposing raw database IDs.
- Apply the principle of least privilege at the data-access layer.

Finding 2 — SQL Injection

Severity: Critical

OWASP Category: A03:2021 – Injection

Description

The `id` parameter is concatenated directly into a SQL query without sanitization or parameterization, allowing arbitrary manipulation of the query logic.

Proof of Concept

```
Step 1 - Confirm injection:
GET /vulnerabilities/sqli/?id=1'&Submit=Submit
-> Response reveals raw SQL syntax error

Step 2 - Extract data via UNION-based injection:
GET /vulnerabilities/sqli/?id=' UNION SELECT user,password FROM users-- -
-> Returns all usernames and password hashes
```

Impact

Full compromise of the application's user database, including credential exfiltration. In production environments, this can escalate to complete data breach or remote code execution depending on database privileges.

Remediation

- Use parameterized queries / prepared statements for all database interactions.
- Apply strict input validation using allowlists.
- Enforce least-privilege database accounts.
- Disable verbose SQL error messages in production.

Finding 3 — Cross-Site Scripting (XSS)

Severity: High
OWASP Category: A03:2021 – Injection

Description

User-supplied input is reflected/stored and rendered in the HTML response without proper output encoding, allowing execution of attacker-controlled JavaScript in victims' browsers.

Proof of Concept

```
GET /vulnerabilities/xss_r/?name=<script>alert(document.cookie)</script>
```

The payload executed successfully in the browser context, confirming the ability to access session cookies and other client-side data.

Impact

Successful exploitation enables session hijacking, credential theft, and account takeover without requiring the victim's password — a single malicious link or stored payload is sufficient.

Remediation

- Apply context-aware output encoding (e.g., `htmlspecialchars()` with `ENT_QUOTES`).
- Implement a strict Content Security Policy (CSP).
- Set `HttpOnly` and `Secure` flags on session cookies.
- Enforce server-side input sanitization; never rely on client-side validation alone.

Finding 4 — Broken Authentication

Severity: High
OWASP Category: A07:2021 – Identification & Authentication Failures

Description

The login mechanism lacks rate limiting and account lockout controls, and session tokens are not regenerated after successful authentication.

Proof of Concept

```
Automated brute-force attempt via Burp Suite Intruder:  
POST /login.php  username=admin&password=$FUZZ$  
-> Thousands of attempts submitted with no lockout or CAPTCHA triggered
```

Impact

Attackers can perform unrestricted brute-force attacks to guess valid credentials, and the lack of session regeneration exposes the application to session fixation attacks, enabling account takeover.

Remediation

- Implement account lockout after a defined number of failed attempts, or use CAPTCHA.
- Regenerate the session ID immediately after successful login.
- Enforce HTTPS/TLS for all authentication traffic.
- Implement Multi-Factor Authentication (MFA) for privileged accounts.

5. Remediation Summary

Finding	Priority	Recommended Fix
SQL Injection	Immediate	Parameterized queries

IDOR	Immediate	Server-side authorization checks
XSS	Short-term	Output encoding + CSP
Broken Authentication	Short-term	Rate limiting + session regeneration + MFA

6. Tools Used

- Burp Suite (Proxy, Repeater, Intruder)
- OWASP ZAP
- sqlmap
- Manual testing techniques

7. Disclaimer

This assessment was conducted in a controlled lab environment against DVWA, an intentionally vulnerable application designed for security training purposes. All testing was performed for educational purposes only. No production systems or real user data were involved.